PICID: PROOF-DRIVEN CLAUSE LEARNING IN NEURAL NETWORK VERIFICATION

November 6, 2026

NEURAL NETWORK VERIFICATION

Motivation

- Goal: check if a neural network meet some specifications (especially important since they are largely black boxes)
- Idea: cast the question as a **formal verification** problem.

Neural network verification community

- VNN-COMP: international competition, yearly since 2020.
- Shared benchmarks and standard formats (ONNX networks, VNN-LIB properties).
- Many competing verifiers: α, β-CROWN, NeuralSAT, Marabou, ...

Why is it hard?

- **Large number of variables**
  - large networks, with many neurons and parameters.
- **Complex non-linearities**
  - ReLU networks: checking is NP-complete;
  - tanh, sigmoid, softmax: involve $\exp \Rightarrow$ decidability **open**.

TYPICAL SPECIFICATIONS

Robustness

Local: $\forall i \in B(i^*, \varepsilon), \quad N(i) \in B(o^*, \delta)$

Global: $\forall i, i', \quad \|i - i'\| \leq \varepsilon \;\Rightarrow\; \|N(i) - N(i')\| \leq \delta$

small perturbations of the input do not change the output much

Safety

$\forall i \in A, \quad N(i) \in B$

inputs in a region of interest never produce dangerous outputs

Substitution robustness

$\forall i \in \Sigma^*$, $\forall \sigma$ substitution:

$\arg\max N(\sigma(i)) = \arg\max N(i)$

e.g. replacing words by synonyms does not change the predicted class

Monotonicity

$s(i) \leq s(i') \;\Rightarrow\; N(i) \leq N(i')$

if a feature s of the input increases, the output cannot decrease

MAIN APPROACHES

1) Linear bound propagation + branch-and-bound

e.g. CROWN, α, β-CROWN

- \+ sound (but not tight) bounds
- \+ adapts to many activation functions
- \+ very efficient (GPU)
- \+ The relaxed bound is differentiable, so it can be optimized during training!
- – symbolic relaxations are hard to turn into checkable proofs

2) Simplex + case splitting

e.g. Reluplex, Marabou

- \+ natural certificates of UNSAT (Farkas lemma)
- – scales less well

3) CDCL(T) hybrids

e.g. NeuralSAT, DeepCDCL

- \+ SAT-solver techniques to guide the case splitting of activations (clause learning, backjumping)

RELIABILITY AND PROOF PRODUCTION

Problem

Verifiers rely on floating-point arithmetic ⇒ they are sometimes **unsound**.

Solution

Proof production: the verifier outputs a proof of its answer, checked separately.

Limits of existing DNN proofs

- high-level proofs, costly to check
- no standard proof format
- the only formally verified checker (Imandra) does not scale
- proof-producing verifiers do not scale

PICID

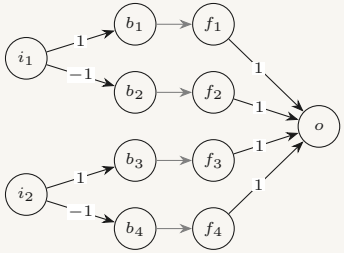
Goal: improve trust and scalability

ENCODING A VERIFICATION QUERY

Variables and constraints

- **Variables**: $x = (i, b, f, o)$: inputs, ReLU inputs / outputs, output
- **Linear equations** $Ax = 0$ (model the weights):
  $b_1 = i_1,\ b_2 = -i_1,\ \ldots,\ o = f_1 + \cdots + f_4$
- **Bounds** $l \le x \le u$:
  - **inputs**: input property (e.g. $i_1 \in [-2, 1]$, $i_2 \in [-2, 2]$)
  - **output**: unsafe region (e.g. $o \ge 6$)
  - **intermediate neurons**: $f_j \ge 0,\ f_j \ge b_j$
- **Activations**: $f_j = \mathrm{ReLU}(b_j)$
- **Problem**: $Ax = 0 \wedge l \le x \le u \wedge \bigwedge_j f_j = \mathrm{ReLU}(b_j)$

  UNSAT: property certified SAT: counterexample
  Here $o = |i_1| + |i_2| \le 4$: UNSAT



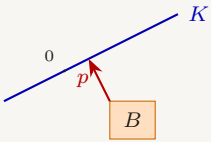
Key idea: relax the activations

- Drop $f_j = \mathrm{ReLU}(b_j)$ and keep only
  $f_j \ge 0,\quad f_j \ge b_j \qquad$ (i.e. $f_j \ge \max(b_j, 0)$)
  $\Rightarrow$ makes the problem **linear**.
- Relaxation UNSAT $\Rightarrow$ unsafe region never reached.
- Relaxation SAT: inconclusive if some ReLU are violated.

FARKAS LEMMA: CERTIFICATES OF UNSAT

Theorem (Farkas lemma)

$Ax = 0,\ l \le x \le u$ is UNSAT $\iff$ $\exists w$ such that $\sum_{c_j>0} c_j u_j + \sum_{c_j<0} c_j l_j < 0$, where $c = w^\top A$



Proof (insight)

$\Leftarrow$ if w exists and x is a solution:

- $0 = w^\top Ax = \sum_j c_j x_j \le \sum_{c_j>0} c_j u_j + \sum_{c_j<0} c_j l_j < 0.$

$\Rightarrow$ the neuron's values $K = \ker A$ and the box B are disjoint:

1. take p the point of $K - B$ closest to 0: then $p^\top z \ge \|p\|^2$ on $K - B$;
2. K is a subspace, so $p \perp K$, i.e. $p^\top = w^\top A$;
3. hence $p^\top x \le -\|p\|^2 < 0$ on the whole box.

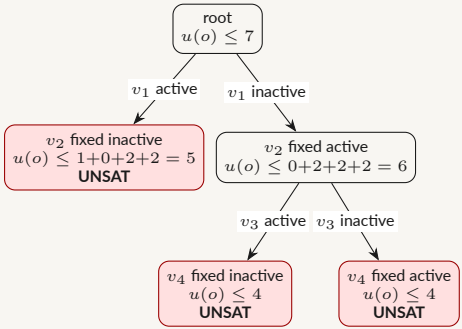
Reading a certificate

- w tells **which bounds are used** (the faces of B meeting at the corner closest to K): u_j if $c_j > 0$, l_j if $c_j < 0$.
- Finding w: an LP. Checking w: cheap, in exact arithmetic.

Main loop

1. Solve the **relaxed** LP (ReLU constraints ignored).
2. **Tighten bounds**; a ReLU with $l(b_j) \geq 0$ or $u(b_j) \leq 0$ is already fixed (active / inactive).
3. **Check at every node**:
   - LP infeasible $\Rightarrow$ **branch closed** (Farkas certificate); continue with the other branches
   - LP solution satisfies all ReLUs $\Rightarrow$ **SAT**: real counterexample, stop
   - otherwise: inconclusive $\Rightarrow$ branch

   final answer UNSAT $\iff$ all branches are closed
4. Otherwise, **branch** on one unstable ReLU ($l(b_j) < 0 < u(b_j)$), and repeat in both cases.



termination: each split fixes at least one more ReLU; once all are fixed, the LP is exact

WHAT IS SUBOPTIMAL IN THIS SEARCH

Limitations

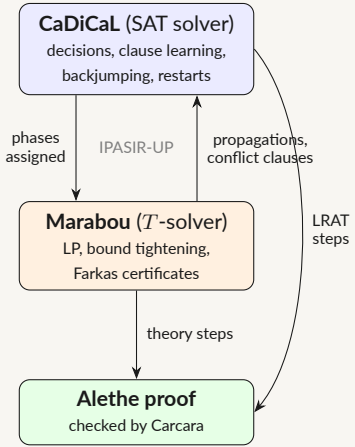
1. **Choice of the branching variable**: a fixed heuristic, which does not learn from past conflicts.
2. **No learning across branches**: the search is depth-first and forgets everything when it backtracks, so an infeasible combination of phases is proved again in every subtree where it appears.
3. **Chronological backtracking**: after a conflict, the search returns to the most recent decision, even if the conflict does not depend on it.
4. **Large proofs**: the Farkas vectors (lemmas) of every branch are recorded, and they are not pruned before the proof is written.

Observation

- These are exactly the problems that **CDCL** solves in modern SAT solvers $\Rightarrow$ **PICID**

What the switch to CDCL(T) brings

- **Learned clauses**: an infeasible combination of activations is never explored again, in any branch.
- **Backjumping**: return directly to the decision responsible for the conflict.
- **VSIDS decisions**: branch first on neurons involved in recent conflicts.
- **Proof-driven minimization**: clauses keep only the decisions actually used by the certificates ⇒ shorter clauses (more pruning) and smaller proofs.
- **Parallelism**: the query is split into subqueries solved by independent workers; the proofs are combined at the end.
- **Standard tools**: state-of-the-art SAT solver (CaDiCaL), standard proof format (Alethe).



Mechanics

- **Boolean abstraction**: one Boolean p_j per ReLU, p_j true $\iff$ ReLU j is active.
- **Decisions**: the SAT solver decides the activation of a ReLU; Marabou tightens the corresponding bounds.
- **Propagation**: activations fixed by bound tightening are sent back to the SAT solver.
- **Conflict**: for each bound used by the Farkas certificate,
  - it comes from a decision: add the negated decision to the clause;
  - it comes from a lemma: recurse on the lemma's own certificate.

  The set of lemmas is minimized: unused lemmas disappear from both the clause and the proof.

On the toy example

Leaf "ReLU 1 active": the certificate uses $u(f_2) = 0$ (ReLU 2 fixed inactive), which comes from the decision p_1
$\Rightarrow$ learned clause $\neg p_1$: ReLU 1 is inactive everywhere

Leaves under "ReLU 1 inactive":
$p_1 \vee \neg p_3$ $\quad$ $p_1 \vee p_3$

Resolution:

$$\dfrac{\dfrac{p_1 \vee \neg p_3 \qquad p_1 \vee p_3}{p_1} \qquad \neg p_1}{\bot}$$

each clause comes with its proof: the final proof is this resolution tree plus the Farkas steps

ADDITIONAL DETAILS

Proofs in Alethe

- Query written in SMT-LIB (QF_LRA); each ReLU as `(ite (>= b 0) (= b f) (<= f 0))`
- CaDiCaL's LRAT steps become resolution steps; Marabou's certificates become arithmetic steps, computed in exact arithmetic (GMP).
- **Holes**: when Marabou's floating-point sanity check rejects a certificate, the step is marked as `hole`, which localizes possible errors.
- Marabou's **preprocessing**; producing proofs for preprocessing is known to be tricky in SMT solvers as well.

Search

- **Decision heuristic**: VSIDS combined with Marabou's own branching score and the set of satisfied clauses (exact formula not given in the paper).
- **Parallelism**: each subquery gets its own SAT solver and proof writer; a final step combines the sub-proofs.

RESULTS

Method	Solved SAT	Solved UNSAT	Shared verif. (s) SAT	Shared verif. (s) UNSAT	Shared check (s)	Validity V/H/I/O/N
ACAS Xu 180 queries, 300 neurons						
Marabou	40	87	327.71	365.89	N/A	N/A
Marabou + Alethe	40	82	467.06	522.06	395.45	82/0/0/0/0
PICID	42	95	241.38	226.46	**113.68**	94/1/0/0/0
PICID + SNC	**45**	110	**59.43**	128.34	287.50	98/1/0/11/0
NeuralSAT	**45**	**135**	101.57	**48.70**	730.58	58/0/0/22/55
CERSYVE 20 queries, 64–198 neurons						
Marabou	10	8	15.04	104.39	N/A	N/A
Marabou + Alethe	10	7	30.32	173.83	103.25	5/2/0/0/0
PICID	10	8	10.56	64.00	**27.02**	5/1/2/0/0
PICID + SNC	10	**10**	**3.86**	**29.68**	27.28	6/2/2/0/0
SafeNLP 1080 queries, 128 neurons						
Marabou	577	209	2.89	60.00	N/A	N/A
Marabou + Alethe	578	202	2.95	93.83	61.82	156/0/0/0/46
PICID	620	246	1.36	18.70	**10.48**	200/0/0/0/46
PICID + SNC	**632**	**263**	**1.30**	**9.18**	14.66	216/0/0/1/46
MNIST FC 540 queries, 512–1024 neurons						
Marabou	35	165	487.97	290.39	N/A	N/A
Marabou + Alethe	36	165	559.33	342.58	8.35	157/0/0/0/8
PICID	25	209	844.81	75.19	**2.90**	201/0/0/0/8
PICID + SNC	**54**	**215**	**149.55**	**50.39**	7.88	207/0/0/0/8

V/H/I/O/N = valid / holey / invalid / out of resources / not generated. “Shared”: averages over queries solved by all methods.

RESULTS

Method	Solved SAT	Solved UNSAT	Shared verif. (s) SAT	Shared verif. (s) UNSAT	Shared check (s)	Validity V/H/I/O/N
ACAS Xu 180 queries, 300 neurons						
Marabou	40	87	327.71	365.89	N/A	N/A
Marabou + Alethe	40	82	467.06	522.06	395.45	82/0/0/0/0
PICID	42	95	241.38	226.46	**113.68**	94/1/0/0/0
PICID + SNC	**45**	110	**59.43**	128.34	287.50	98/1/0/11/0
NeuralSAT	**45**	**135**	101.57	**48.70**	730.58	58/0/0/22/55
CERSYVE 20 queries, 64–198 neurons						
Marabou	10	8	15.04	104.39	N/A	N/A
Marabou + Alethe	10	7	30.32	173.83	103.25	5/2/0/0/0
PICID	10	8	10.56	64.00	**27.02**	5/1/2/0/0
PICID + SNC	10	**10**	**3.86**	**29.68**	27.28	6/2/2/0/0
SafeNLP 1080 queries, 128 neurons						
Marabou	577	209	2.89	60.00	N/A	N/A
Marabou + Alethe	578	202	2.95	93.83	61.82	156/0/0/0/46
PICID	620	246	1.36	18.70	**10.48**	200/0/0/0/46
PICID + SNC	**632**	**263**	**1.30**	**9.18**	14.66	216/0/0/1/46
MNIST FC 540 queries, 512–1024 neurons						
Marabou	35	165	487.97	290.39	N/A	N/A
Marabou + Alethe	36	165	559.33	342.58	8.35	157/0/0/0/8
PICID	25	209	844.81	75.19	**2.90**	201/0/0/0/8
PICID + SNC	**54**	**215**	**149.55**	**50.39**	7.88	207/0/0/0/8

V/H/I/O/N = valid / holey / invalid / out of resources / not generated. “Shared”: averages over queries solved by all methods.